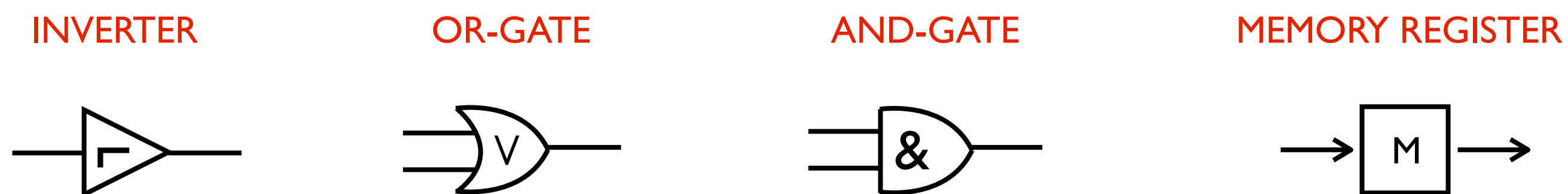


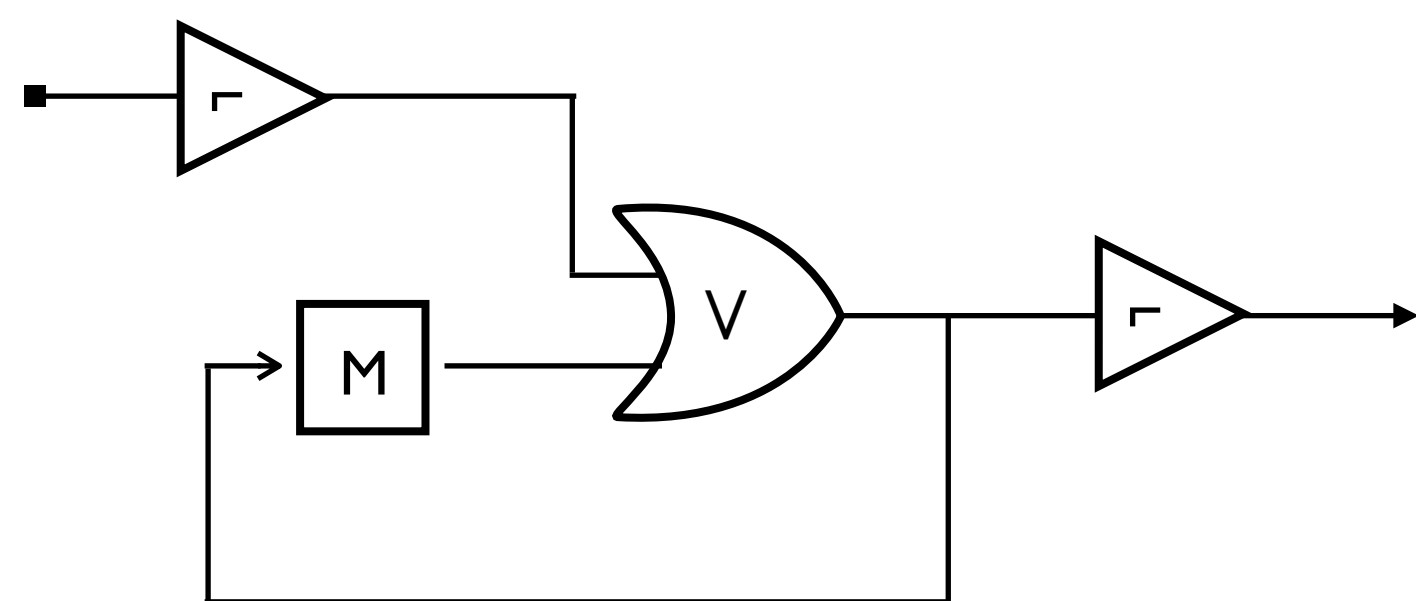
I 9. Machines in Time: Sequential Circuits

Sequential Circuits = Combinatorial Circuits + Memory

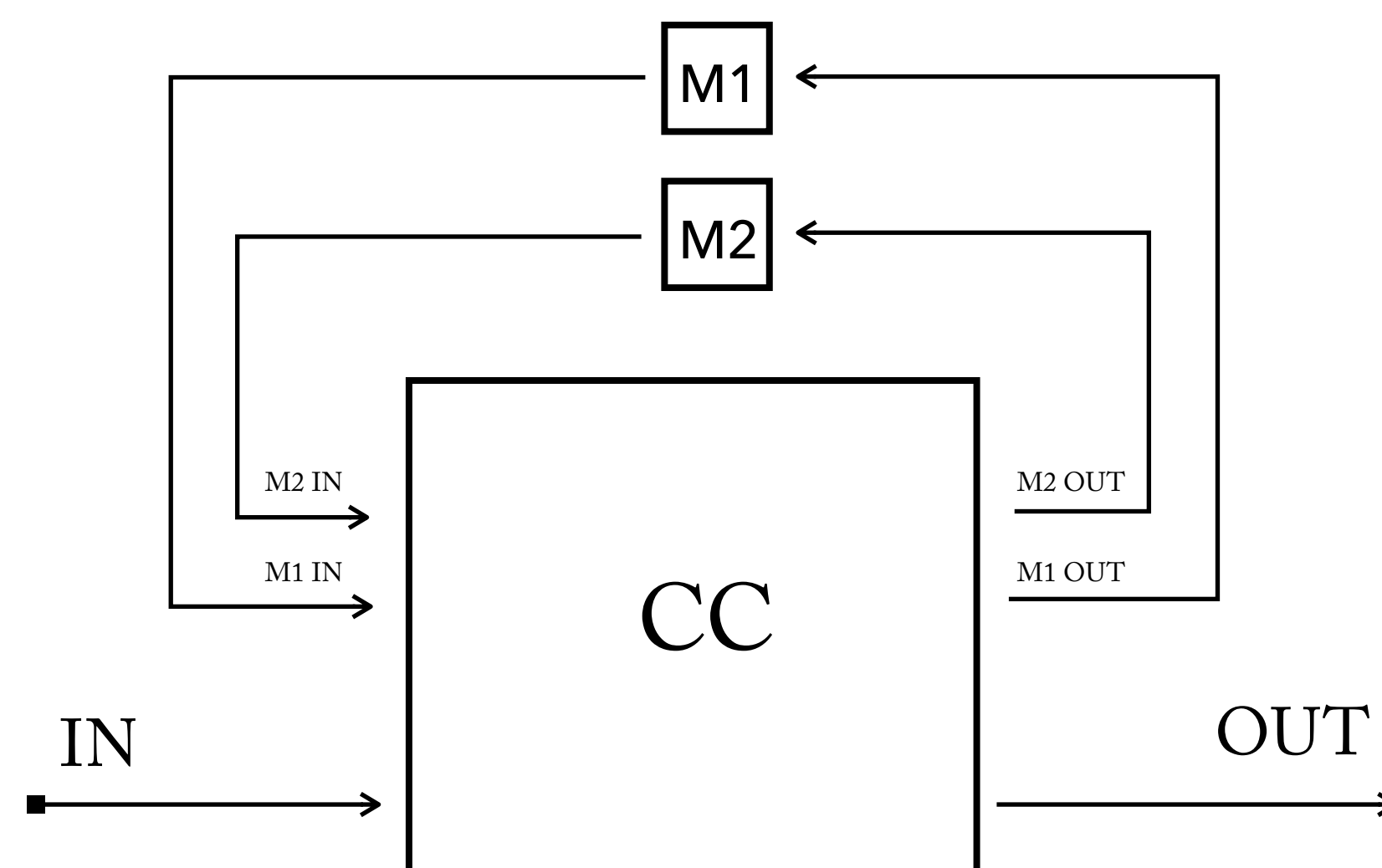
A **sequential circuit** (SC) is a combinatorial circuit combined with a finite number of memory registers. Our circuits now consists of four basic parts.



The structure of a sequential circuit is just like a combinatorial circuit, except that **loops are allowed**, but **only when they are interrupted by a memory register**. Like this:

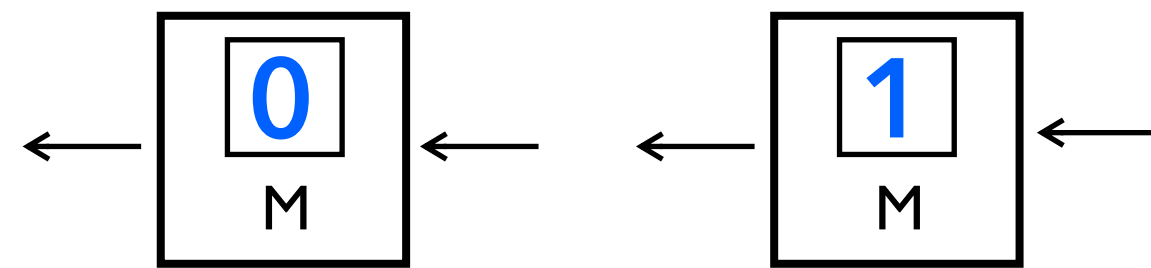


In a **standard layout**, the memory registers are kept separate from the combinatorial circuit. The inputs to the circuit as a whole are aligned with inputs to the CC from memory; the outputs of the circuit as a whole are aligned with the output from the CC to memory. This design that makes the conceptual relationship between CC and memory visually clear. (You don't have to use standard layout when designing circuits.)

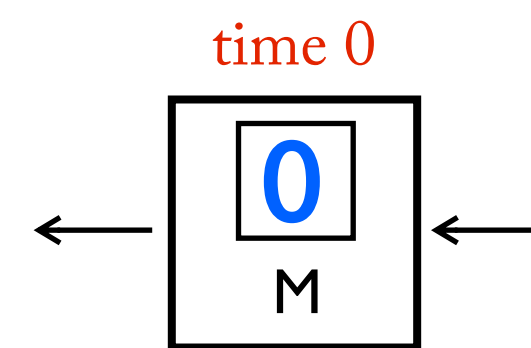


Memory Registers

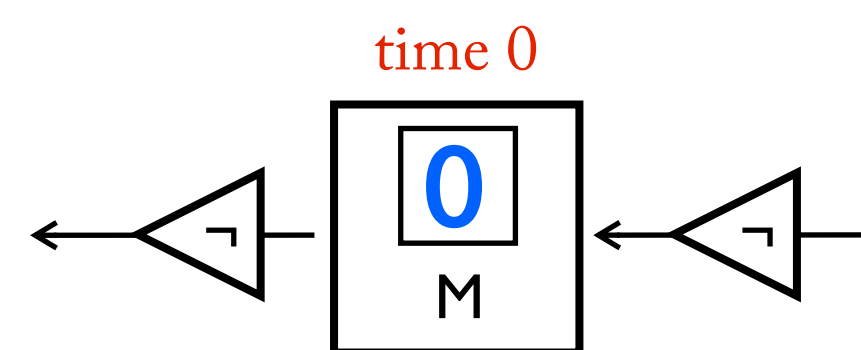
Each memory register can store exactly one 1-bit.



We'll assume our memory registers are **0-first**:
at time 0 they always begins by storing a 0.



But you can build a **functional 1-first** memory with a circuit that uses a 0-first memory, but simulates the behavior of 1-first memory. It outputs a 1 on the first cycle



The Clock

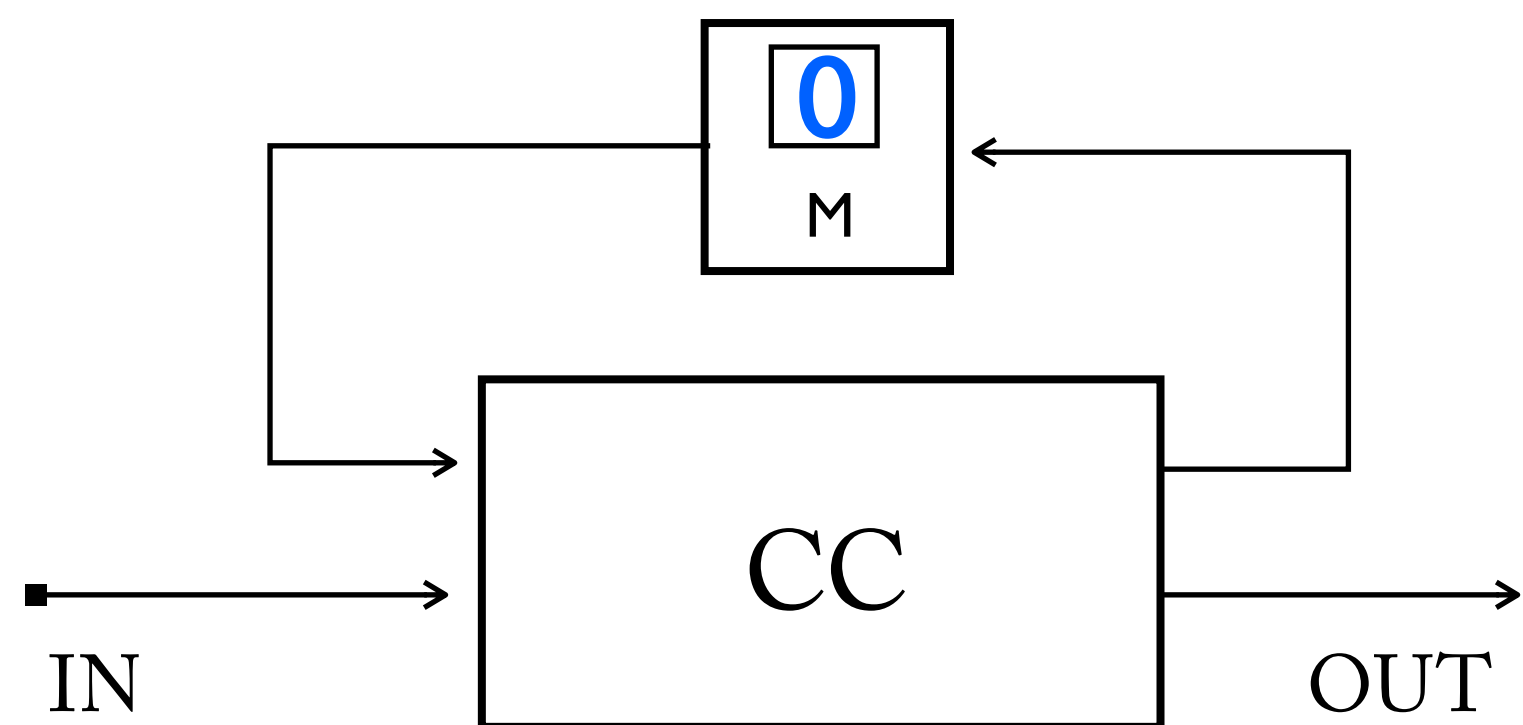
Machine time is governed by an abstract clock, which goes through discreet cycles. At the beginning of each cycle, inputs are read, and output are written or displayed. As before, within a cycle, the machine works instantaneously.



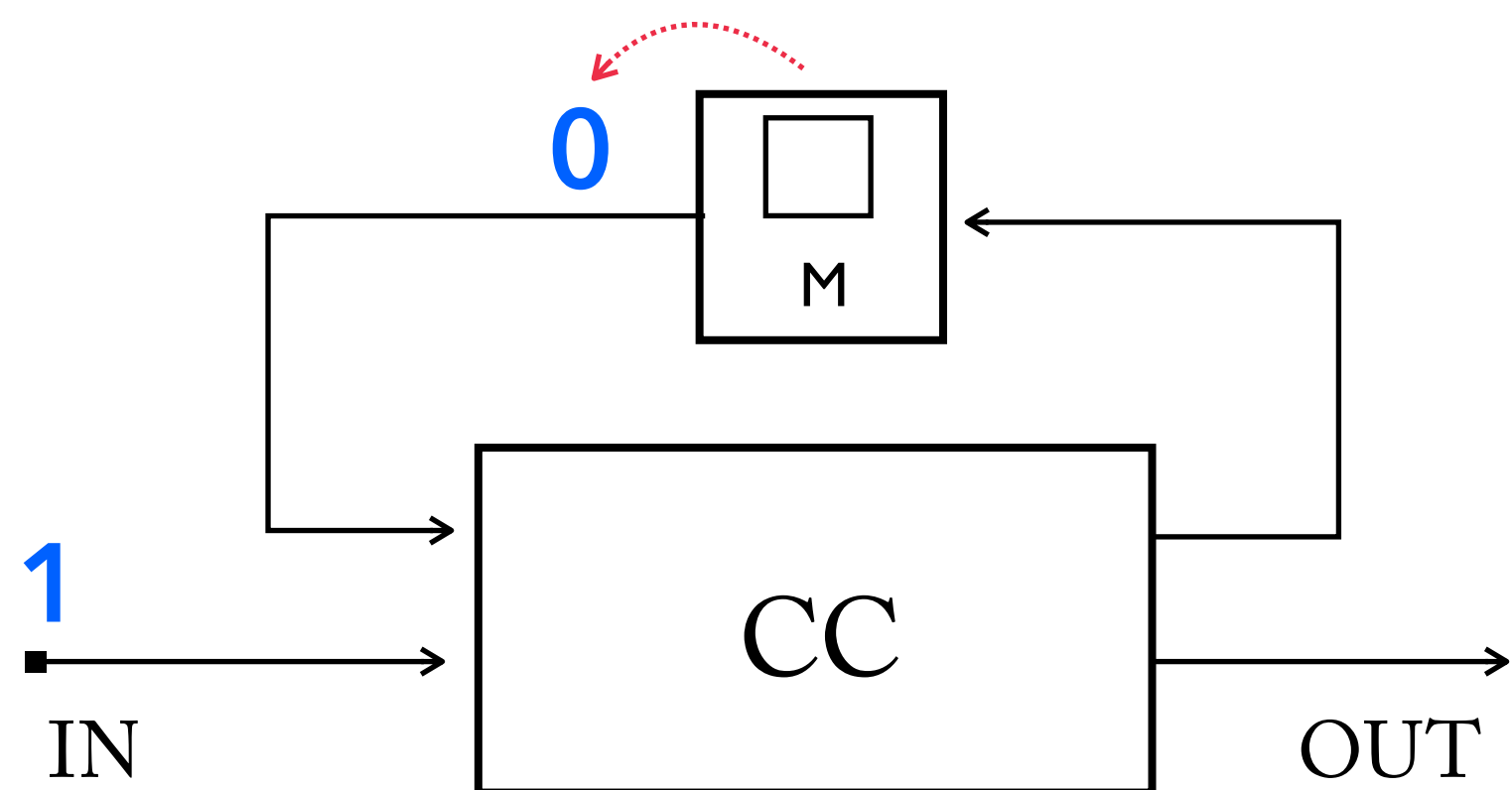
What makes memory registers *memory*— in other words, what makes them carry information forward through time— is that they preserve the bit they carry after each cycle of the clock.

Cycles of a Sequential Circuit

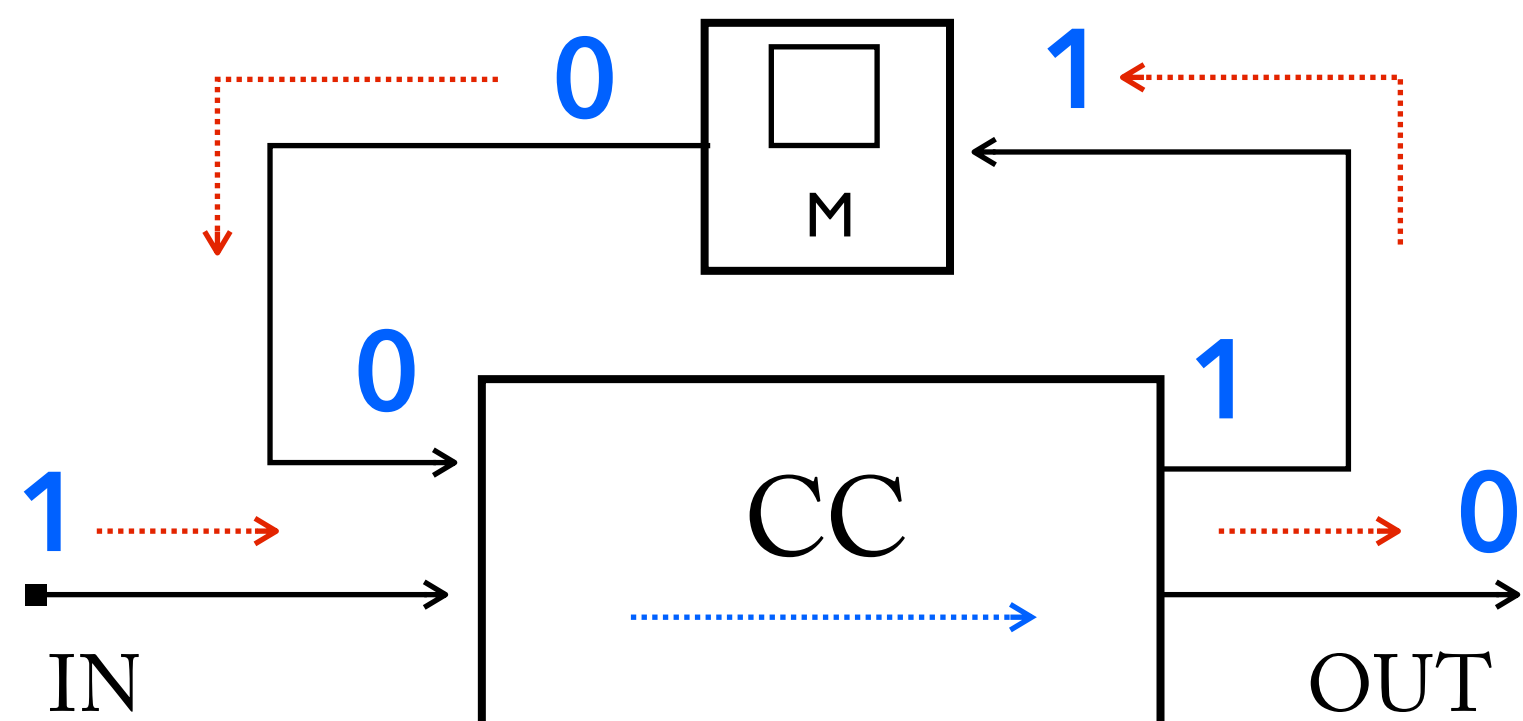
1. In the beginning, the circuit is quiet and a bit is stored in each memory register.



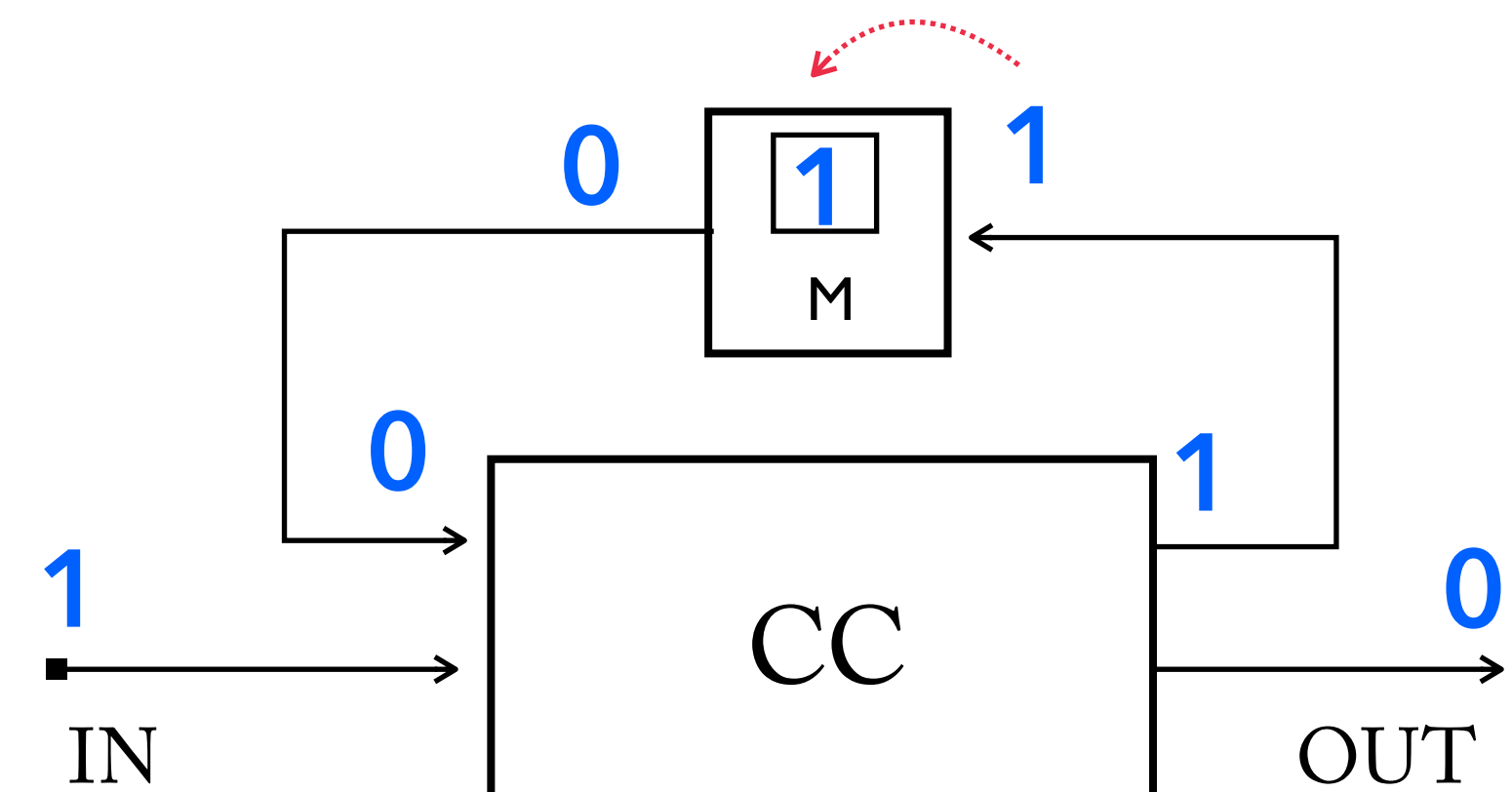
2. Input is received from each local input and from each memory register.



3. Run the circuit, produce outputs.

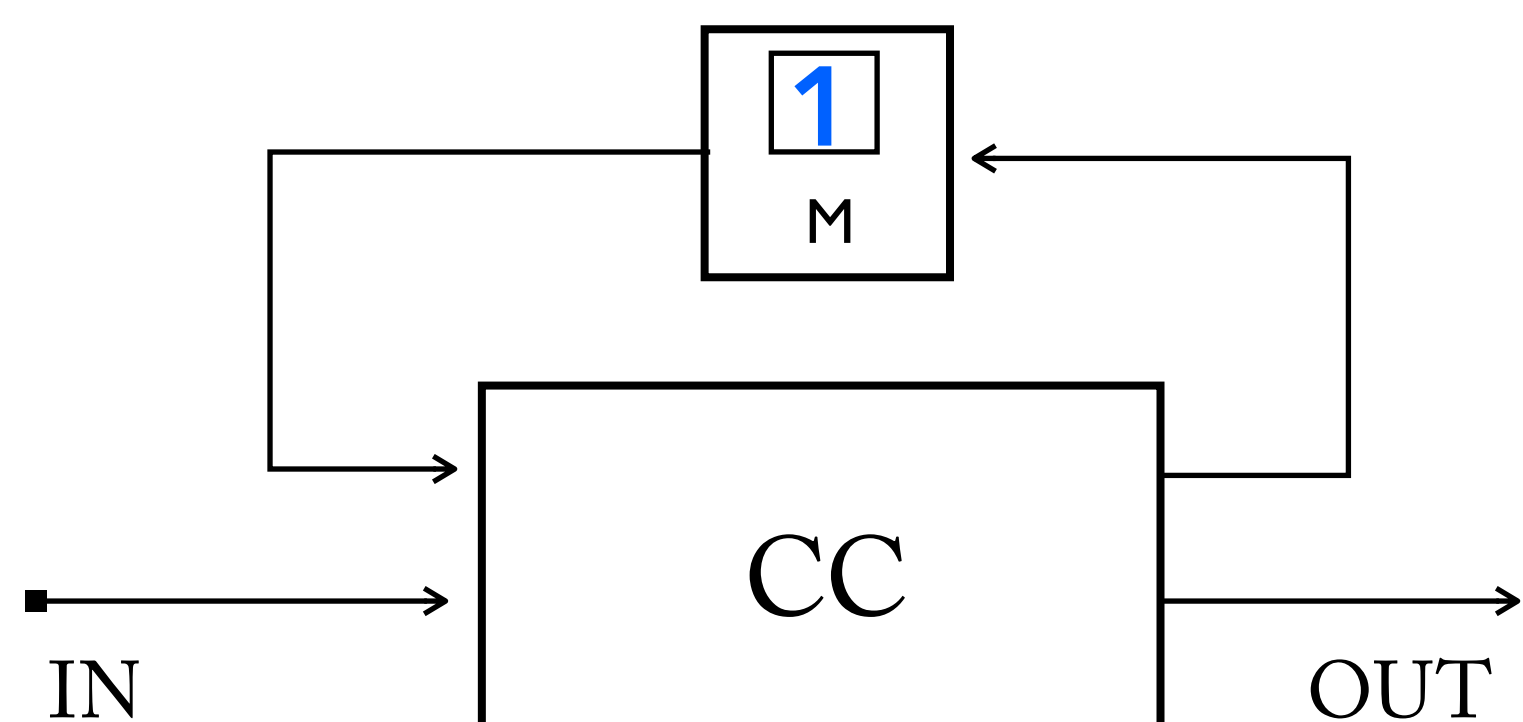


4. Record new bits into memory registers.



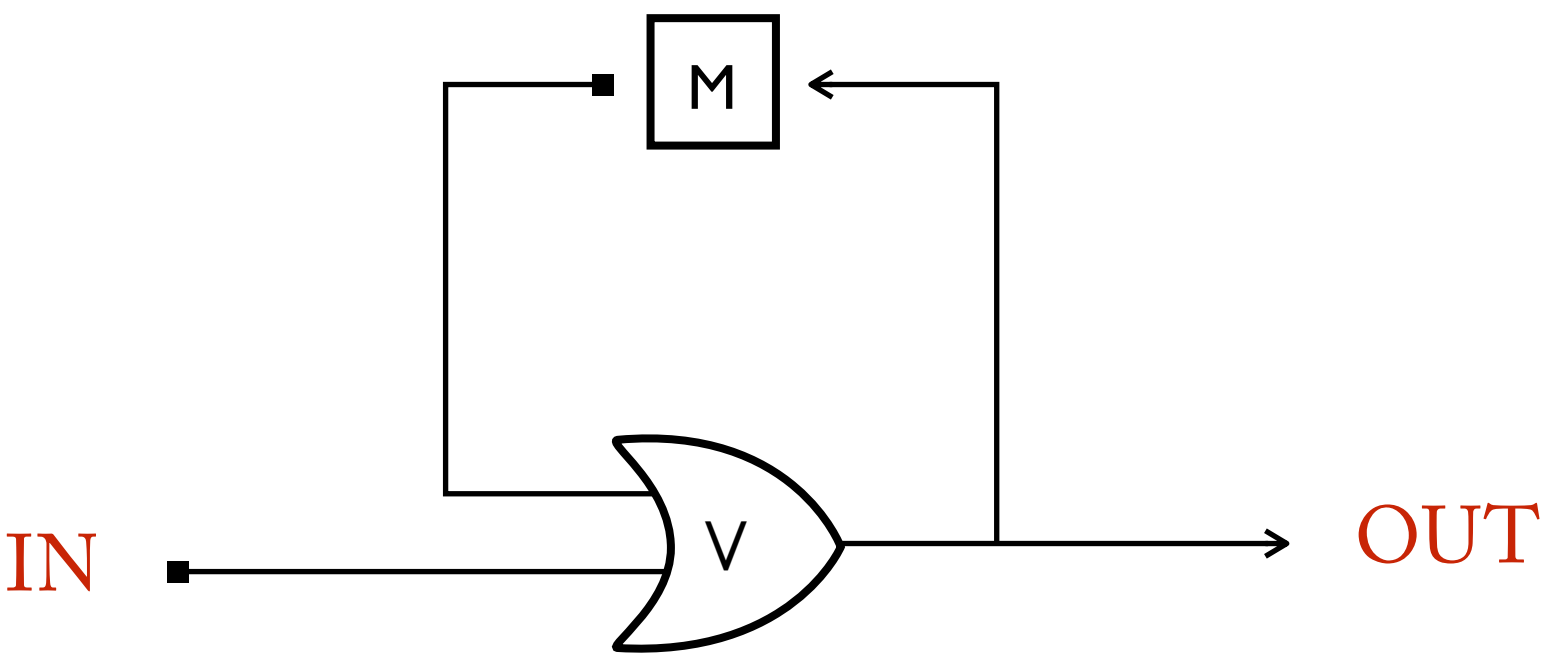
5. Clear the circuit of all information except for the memory circuits.

Initiate the next cycle.



Memoirs of a Sequential Circuit

To keep track of the behavior of a sequential circuit, we'll typically have to keep *two* kinds of tables: one **global input-output table**, and a set of **sequential local input-output tables**.



Global input-output table

IN	OUT
000	000
001	111
010	
100	100
100	
101	111
...	...

Sequential local input-output tables

...	t5	t4	t3	t2	t1	
...	0	0	0	0	0	IN
...	0	0	0	0	0	OUT

...	t5	t4	t3	t2	t1	
...	0	0	0	0	1	IN
...	1	1	1	1	1	OUT

...	t5	t4	t3	t2	t1	
...	0	0	0	1	1	IN
...	1	1	1	1	1	OUT

...	t5	t4	t3	t2	t1	
...	0	0	1	0	1	IN
...	1	1	1	1	1	OUT

What does this circuit *do*?
 At which point in the sequence does this machine exhibit distinctively sequential history-dependent behavior?

In-Class Problems

Design SC's which solve the following problems.

1. **Temporal AND:**

Output = 1 iff the current input = 1 **and** the last input = 1.

2. **Temporal OR:**

Output = 1 iff the current input = 1 **or** the last input = 1.

3. **Trip-wire**

Output = 0 iff there is or ever was an input = 0.

4. **Pattern Finder:**

Output = 1 as long as the input alternates 1's and 0's (e.g. "101010..."). If the pattern is ever broken, output = 0 forever after.

5. **ON/OFF switch:**

Starts in the OFF state. If you push a button, it goes ON. If you push the button again, it goes OFF. (**Hint:** think in tables.)